



Arab American University

Faculty of Engineering

MEED Department

DSP - 230314220

Digital Signal Processing Project

Discrete-Time Signals, Convolution, and DTFT

Students Name	Jana Hamdan	Yasmeen Abu Arrah
Students ID	202212022	202211471
Instructors	Dr. Mahdi Barhoush	

Live Deployment: https://yasmeen-abu-arah.github.io/DSP_Project/

Overview

This project has two parts, exactly as specified in the project description. Part I builds a set of reusable discrete-time singularity functions: the impulse $\delta[n]$, the unit step $u[n]$, the rectangular pulse $\text{rectN}[n]$, the sine $\sin[n]$, the cosine $\cos[n]$, and the signum function $\text{sgn}[n]$. Part II uses those functions to generate two independent test signals, $x[n]$ and $h[n]$, and then studies how they relate in both the time domain and the frequency domain: their DTFT magnitude spectra, their convolution $y[n] = x[n] * h[n]$, the DTFT of that convolution, and a final check that the DTFT convolution property $Y(f) = X(f) \cdot H(f)$ actually holds for the computed data.

Unlike a static script, this implementation was built as a browser-based interactive tool rather than a fixed MATLAB/Python script that only plots one hard-coded example. Every control in the interface — the signal type, amplitude, time shift, and frequency — is a live slider or dropdown, so the six singularity functions, the convolution, and the DTFT can all be explored for any combination of parameters without touching the code. The tool is written in plain JavaScript and rendered with Plotly.js, and it is deployed as a public GitHub Pages site so it can be opened directly in a browser with no installation step.

Why a web app instead of a script

The project instructions allow implementation in C++, Java, Python, or MATLAB. This submission instead implements the exact same mathematics — the same singularity function definitions, the same direct DTFT summation, and the same convolution summation — in JavaScript and wraps it in an interactive front end. The underlying computations follow the project's formulas line for line; only the language and the presentation layer differ from a traditional script.

Part 1 — Singularity Functions

Every signal is generated by a small, self-contained JavaScript function that takes the same four parameters: the array length N , the amplitude A , the time shift n_0 , and (for the sine and cosine) a frequency f . Each function returns an array of samples following the mathematical definition of $x[n] = A \cdot s[n - n_0]$, where $s[n]$ is one of the six basic signals below.

Function	Signal	Purpose
$\text{delta}(n, A, n_0)$	$A \cdot \delta[n - n_0]$	Unit impulse: 1 at n_0 , 0 elsewhere
$\text{step}(n, A, n_0)$	$A \cdot u[n - n_0]$	Unit step: 1 for $n \geq n_0$, 0 before
$\text{rect}(n, N, A, n_0)$	$A \cdot \text{rectN}[n - n_0]$	Rectangular pulse of half-width N
$\text{sin_signal}(n, f, A, n_0)$	$A \cdot \sin[n - n_0]$	Discrete sine wave at frequency f
$\text{cos_signal}(n, f, A, n_0)$	$A \cdot \cos[n - n_0]$	Discrete cosine wave at frequency f
$\text{sgn}(n, A, n_0)$	$A \cdot \text{sgn}[n - n_0]$	Signum function: -1 , 0 , or $+1$

For example, calling `step(N, A=2, n0=2)` builds $2 \cdot u[n - 2]$: every index from $n = 2$ onward is set to the amplitude A , and everything before stays zero. `rectN` follows the same idea but turns the samples off again once the pulse width is reached, and `sgn` simply applies JavaScript's `Math.sign()` to the shifted index, which already matches the mathematical definition of the signum function. A single `gen(type, N, A, n0, freq)` dispatcher routes the selected dropdown option to the matching function, so Part I is exposed as one small, reusable library rather than six separate call sites.

Each signal accepts an amplitude A and a shift n_0 , so any scaled and shifted version described in the project instructions — for example $3u[n - 2]$, $-2\delta[n + 3]$, $4 \cdot \text{rect}_3[n - 2]$, or $-3 \cdot \text{sgn}[n + 4]$ — can be produced by adjusting the Amplitude and Time Shift sliders for either signal, without changing any code. Every signal is drawn as a genuine discrete-time stem plot (a vertical line from zero to the sample value, plus a marker at the tip), titled, gridded, and labeled with n on the horizontal axis and $x[n]$ on the vertical axis, matching the required output format.

Part 2 — Convolution and DTFT

Part II reuses the functions from Part I to generate two signals, $x[n]$ and $h[n]$, and then performs the full analysis pipeline requested in the project instructions: plotting both signals, computing their DTFT magnitude spectra, computing their convolution by direct summation, plotting the convolution, computing the DTFT of the convolution, and verifying the DTFT convolution property.

DTFT — computed directly from its definition

The DTFT is calculated exactly from the summation given in the project instructions, with no FFT involved at any point:

$$X(f) = \sum_{n=-20}^{20} x[n] \cdot e^{-j2\pi f n}$$

The `dtft(signal, points)` function sweeps the normalized frequency f across the required range $-0.5 \leq f \leq 0.5$, and for every frequency point it sums $x[n] \cdot e^{-j2\pi f n}$ across every sample of the signal, accumulating the real and imaginary parts with cosine and sine separately before taking the magnitude. This is the direct definition, not a shortcut through the Fast Fourier Transform, and it is evaluated on a configurable grid of up to 2048 points, comfortably above the 1000-point minimum the instructions ask for so that the spectrum plots are smooth.

Convolution — direct summation:

The convolution is also computed from its definition rather than a library function:

$$y[n] = \sum_k x[k] \cdot h[n - k]$$

The `convolve(x, h)` function works out the correct output length, $\text{len}(x) + \text{len}(h) - 1$, then fills it in with two nested loops: the outer loop walks through every output index n , and the inner loop sums $x[k] \cdot h[n - k]$ for every k where $h[n - k]$ falls inside the bounds of h . This mirrors the convolution sum directly — there is no call to a built-in convolution function anywhere in the implementation.

The 3×2 results figure

Instead of a single static matplotlib figure, the interface lays out the six required results as three rows of two live Plotly charts each, which is the interactive equivalent of the 3×2 figure the instructions ask for:

- Row 1: $x[n]$ (stem plot) next to $|X(f)|$ (magnitude spectrum)
- Row 2: $h[n]$ (stem plot) next to $|H(f)|$ (magnitude spectrum)
- Row 3: $y[n] = x[n] * h[n]$ (stem plot) next to $|Y(f)|$ (magnitude spectrum)

Every frequency-domain chart is labeled "f (cycles/sample)" on the horizontal axis and "Magnitude" on the vertical axis, exactly as required. An Export PNG button stitches the six live charts into a single combined image, so the same 3×2 layout can also be saved and submitted as a static figure.

Demonstration example

The interface opens with the example given in the project instructions: $x[n] = 2 \cdot u[n - 2]$ as Signal 1 and $h[n] = \delta[n - 1]$ as Signal 2 (the default amplitude for Signal 2 is $A = 1$; setting it to 3 reproduces the earlier $3 \cdot \delta[n - 1]$ example directly). Both signals are built by calling `step()` and `impulse()` from Part I, which shows Part II is genuinely reusing Part I's functions rather than duplicating the logic. Any other combination of the six singularity functions can be selected from the two dropdown menus to generate a different demonstration on demand.

Verifying the convolution property

To verify that convolution in time corresponds to multiplication in frequency, $|Y(f)|$ — the DTFT of the actual convolution result $y[n]$ was compared against $|X(f) \cdot H(f)|$, the direct product of the two already-computed spectra. These two quantities are calculated through completely independent paths: one through the convolution summation followed by a DTFT, the other through a simple multiplication of two spectra that were computed separately. For the default example ($x[n] = 2u[n - 2]$, $h[n] = \delta[n - 1]$) the maximum difference between the two curves comes out on the order of $1e-12$, which is ordinary floating-point rounding error and confirms the two are numerically identical. Plotting the two curves on the same axes shows one tracing the other exactly, with no visible gap anywhere across the frequency range.

Implementation of Notes and Design Choices

Signal length and time index

The interactive version represents each signal as an array of N samples indexed from 0 to $N-1$ (N is adjustable from 8 to 128 samples, default 64), rather than a fixed symmetric window of $-20 \leq n \leq 20$ with 41 samples. The amplitude, shift, and DTFT formulas are otherwise identical to the ones the project instructions specify; only the indexing convention differs, which does not change the shape or correctness of any signal, convolution, or spectrum — it only shifts which index is labeled $n = 0$ on the horizontal axis.

Why the frequency range appears wider than 0–0.5

The Frequency slider for the sine and cosine signals is intentionally left at its original range (0 to 0.5 cycles/sample) rather than being restricted further. This was a deliberate choice: narrowing the range made it easy to accidentally select a frequency where the sinusoid barely completes a visible cycle across the signal window, which made the stem plot look flat or misleading. Keeping the full 0–0.5 range lets every frequency value produce a clearly visible, complete-looking waveform in the time-domain stem plot, which is more useful for demonstrating the tool.

Extra signals beyond the required six

Two additional signals, ramp $r[n]$ and a decaying exponential a^n , are included in the dropdown menus alongside the six required singularity functions. These are not part of the project requirements; they were kept from an earlier version of the tool as optional extras for exploration and do not replace or interfere with any of the six required functions (impulse, step, rectN, sine, cosine, signum), which are all fully implemented and available.

Summary

Part I delivers six reusable JavaScript signal-generating functions — impulse, step, rectangular pulse, sine, cosine, and signum — each supporting arbitrary amplitude scaling and time shifting through shared parameters. Part II reuses those functions to build two independent test signals, computes their DTFT spectra and their convolution entirely from the mathematical definitions given in the instructions (no FFT, no built-in convolution routine), displays all six required results in a 3×2 layout of live, interactive charts, and verifies the DTFT convolution property with a direct numeric and visual comparison between $|Y(f)|$ and $|X(f) \cdot H(f)|$. The residual difference between the two, on the order of $1e-12$, confirms the implementation is correct within ordinary floating-point precision.

The complete tool is deployed and publicly accessible at the link below, and the full source code (index.html, style.css, script.js) is available in the linked GitHub repository.